

Introduction to Machine Learning (SS 2026)

Programming Project

Author 1

Last name: Mehus
First name: Martin
Matrikel Nr.: 52510699

Author 2

Last name: Edenhauser
First name: Ayleen
Matrikel Nr.: 12408400

I. INTRODUCTION

The goal of this project is to build a system that automatically recognizes static hand gestures for the English alphabet and numeric digits. We formulate this as a multi-class classification task: given an input image, the model must assign it to one of 36 classes, covering the digits 0–9 and the lowercase letters a–z.

The dataset consists of single-channel grayscale images of these sign language gestures. Using grayscale input restricts the model to the raw spatial features and shapes of the hands within a single color channel. To enable a reliable evaluation of the final model, the data was split into 70% for training, 15% for validation, and 15% for a held-out test set.

The dataset consists of 9680 grayscale images of size 128×128 pixels, distributed across the 36 target classes. The class distribution is unbalanced, ranging from 104 to 560 images per class. Digits and a subset of letters (0, 4, 6, 9, c, g, l, p, u, z) are overrepresented relative to the remaining classes. The full per-class distribution is available in `labels.csv`.

II. IMPLEMENTATION / ML PROCESS

A. Data Pre-processing

To improve generalization and reduce overfitting to the training set, online data augmentation was applied using the `torchvision.transforms` module. During each training epoch, images undergo a random rotation of up to 10 degrees (`RandomRotation`) and a random affine translation of up to 10% horizontally and vertically (`RandomAffine`). Since the hand gesture images have a white background, the fill value was set to 255 so that empty regions introduced by rotation or translation remain consistent with the background. Images are loaded as single-channel grayscale matrices using OpenCV, and their dimensions are expanded to match the expected input shape of the network.

B. Model Architecture

A Convolutional Neural Network (CNN) was chosen for this task due to its suitability for computer vision problems. Unlike fully connected networks, which flatten the image immediately and thereby discard spatial structure, a CNN

operates on local regions of the input and is therefore able to learn edges, textures, and the overall shape of the hand gestures.

The network consists of two main components:

- **Feature extractor:** Three sequential convolutional blocks extract features from the input images. Each block consists of a 2D convolutional layer (`Conv2d`) with a 3×3 kernel and padding of 1, followed by a Batch Normalization layer (`BatchNorm2d`) for training stability, a ReLU activation for non-linearity, and a Max Pooling layer (`MaxPool2d`) with a 2×2 window that halves the spatial resolution. The number of channels increases from $1 \rightarrow 32 \rightarrow 64 \rightarrow 128$ across the three blocks.
- **Classifier:** A `Flatten` layer converts the resulting 2D feature maps into a 1D vector of size $128 \times 16 \times 16$. This vector is passed through a series of Linear layers ($32768 \rightarrow 256 \rightarrow 128 \rightarrow 36$), separated by ReLU activations. Dropout layers ($p = 0.15$) are inserted between the linear layers to reduce over-parameterization and prevent memorization of the training data.

C. Hyperparameters

The model was trained using the Adam optimizer with an initial learning rate of 0.001 and a batch size of 128. A `ReduceLROnPlateau` scheduler was used to monitor validation loss and reduce the learning rate by a factor of 0.5 whenever validation loss failed to improve for 2 consecutive epochs. Cross-Entropy Loss was used with a label smoothing factor of 0.1, which prevents the model from becoming overconfident in its predictions and encourages better generalization.

Hyperparameters were selected through manual tuning rather than an automated search. Starting from standard default values for CNN training (e.g., a learning rate of 0.001 for Adam), individual parameters such as batch size, dropout probability, and the learning rate schedule were adjusted iteratively based on the resulting validation loss across training runs, and the configuration yielding the lowest validation loss was kept for the final model.



Fig. 1. Sample predictions on two randomly sampled validation batches. Correct predictions are marked in green, misclassifications in red.

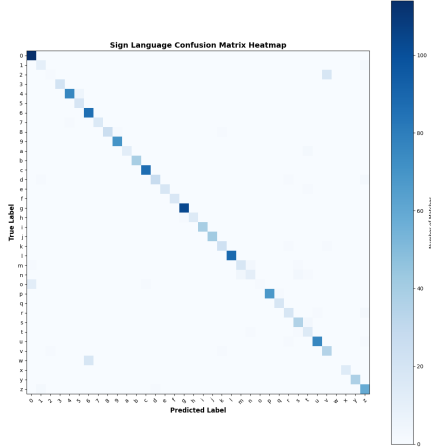


Fig. 2. Confusion matrix heatmap over all 36 classes on the validation set.

To avoid excessive training time and overfitting, an early stopping mechanism was implemented that monitors validation loss and halts training if no improvement occurs for 10 consecutive epochs. Whenever a new lowest validation loss is reached, the corresponding model weights are saved as `best_model.pth`, ensuring that the final deployed model corresponds to the best-performing checkpoint on unseen data.

III. RESULTS

Model performance is illustrated through sample prediction galleries and an overall confusion matrix heatmap.

To evaluate performance on individual characters, two prediction galleries were generated from randomly sampled validation batches (Fig. 1). In these plots, correct predictions are marked in green and errors are marked in red. As shown in the left panel, the network correctly identifies complex hand orientations, including different finger counts and fine structural distinctions such as the letters ‘a’, ‘b’, and ‘x’. In the right panel, nearly all predictions are correct, with one notable exception: the network misclassifies the gesture for the digit ‘2’ as the letter ‘v’.

To assess the distribution of predictions across all 36 classes, a confusion matrix heatmap was generated (Fig. 2). The pronounced diagonal running from the top-left to the bottom-right indicates that the large majority of images

across all alphanumeric categories are classified correctly. The matrix does, however, reveal a small amount of off-diagonal dispersion. For instance, the digit ‘2’ is occasionally confused with letters that involve similar upright finger positions, and minor overlap is visible between visually similar gesture pairs such as ‘0’ and ‘o’. Overall, the heatmap indicates that the feature extraction blocks successfully capture the spatial characteristics needed to distinguish between classes.

IV. DISCUSSION

The high classification accuracy and the clean diagonal of the confusion matrix indicate that the architectural choices made in this project were effective. Applying Batch Normalization after every convolutional layer proved important for stabilizing training and accelerating convergence. In addition, online data augmentation encouraged the network to learn invariant geometric structures rather than relying on fixed pixel alignments. The minor misclassifications observed in the results, such as confusing ‘2’ with ‘v’, are consistent with the visual similarity of the two gestures: both involve two extended fingers held upright, which are difficult to distinguish reliably in a single grayscale channel. Label smoothing of 0.1 was particularly useful in this context, as it prevented the network from assigning excessive confidence to these borderline cases and allowed it to remain more adaptable.

During the initial implementation phase, simpler architectures were tested, including a basic multi-layer perceptron (MLP) and a shallow CNN without batch normalization or dropout. The fully connected MLP performed poorly under spatial translations and failed to capture hand boundaries when the input was tilted. The shallow, unregularized CNN suffered from severe overfitting, with training loss approaching zero while validation loss increased. These observations motivated the introduction of early stopping with a 10-epoch patience threshold and an adaptive learning rate scheduler, both of which noticeably stabilized training.

Several directions could further improve the results. First, network capacity could be increased using a deeper residual architecture (e.g., a mini-ResNet), allowing features to bypass layers and capture more detailed hand structure. Second, additional augmentation strategies such as slight Gaussian blurring or random contrast adjustment could be tested to simulate varying camera and lighting conditions. Finally, a systematic hyperparameter search—for example, tuning batch size or reducing the dropout probability—could yield further performance gains.

V. CONCLUSION

The final model achieved an accuracy of 93.18% (0.9318) on the training set and 88.07% (0.8807) on the held-out evaluation leaderboard, which constitutes the test-set performance for this project. The small gap between these two metrics indicates that the regularization strategies employed were effective, as the model avoided overfitting and generalized well to unseen sign language images.

The central conclusion of this project is that building an effective deep learning model requires more than stacking convolutional layers. Through a carefully designed pipeline—incorporating targeted data augmentation, Batch Normalization for training stability, label smoothing to prevent overconfident predictions, and early stopping to avoid overfitting—a robust gesture classifier was trained from scratch. This process provided practical insight into how these optimization components interact, offering a solid foundation for future computer vision work.